

Theme - Notion Track

1. The Problem

Every college, club, shop, and small agency in India has three or four jobs a person redoes by hand every week. Attendance registers. Requests dying in a WhatsApp group. Form responses copied into a sheet. A follow-up nobody sent. A PDF forwarded to seven people so one of them can retype it.

None of it is hard to automate. It just never got built, because the people doing the job cannot code and the people who can code never see the job. Off-the-shelf tools do not fit, because the job is specific to that college, that shop, that team.

You are not fixing India in a week. You are killing one of those jobs properly and leaving behind a running service plus a workspace the humans can operate after you are gone.

2. What You Are Building

Build a service that automates one real job, with Notion as its interface.

Think of it like this:

Your code is the engine. Notion is the interface. A trigger fires, your code does the work, a real action happens in the outside world, and a row lands in the Run Log. The human never runs anything. They read what happened, approve what matters, and override what is wrong.

The three things your system must do

1. **It runs without you.** A webhook, a cron, or an inbound event fires it, on something you deployed. Running a script by hand during your demo is not a running service.
2. **Humans approve the decisions that matter, inside Notion.** At least one point in your workflow pauses and waits for a person to approve, reject, or override before the action fires.
3. **It leaves proof.** Every run writes a row to the Run Log with a real timestamp, written by your code. Rows spread across the event, not sixty rows created the night before demo day.

What you are NOT building

- A Zapier chain with a Notion page sitting on top of it. If the interesting part of your system lives inside a no-code canvas, you are in the wrong track.
- A chatbot. Chat is a doorway, not a system.
- A dashboard full of charts with no engine behind it.
- A React app that treats Notion as a database and gives the human nothing. A person has to be able to do their whole part of the job inside Notion.
- Five shallow features. One job killed cleanly beats five half-wired ideas.

<aside>



The test that kills the shortcut: delete your repo. Does the system still work?

If yes, you did not build anything. You wired up a no-code tool and put a Notion page on top. That does not score.

</aside>

3. How the System Flows

Any stack, any framework. The shape below is the reference, not a requirement.

flowchart LR

```
T["⚡ Trigger<br/>webhook, cron, inbound event"]
A["💻 Your service<br/>your repo, your host, your logic"]
X["🌐 Real action<br/>message sent, file made, API called"]
R["📄 Run Log<br/>written by your code"]
H["👤 Human approval<br/>only when it needs a person"]

T --> A
A --> X
A --> R
A -.->|"stuck or risky"| H
H -->|"person decides"| A
```

Three ideas to internalize from this diagram:

- **The engine is your code.** The logic that decides what happens lives in a repo you can show us. Your code talks to Notion through the API or the MCP server, with your own integration token.

- **Notion is the interface, not the middleware.** It is where the data lives, where humans approve, and where every run gets logged. Do not route every step through it.
- **The action happens outside Notion.** A message sent, a file made, an API called. If nothing changes in the real world, you built a dashboard.

4. Where AI Actually Earns Its Place

AI handles what rules cannot: reading messy input, sorting it, drafting the action. Your code is the one calling it. An AI property inside a Notion database is not an engine.

Earns it:

- A form arrives as one messy paragraph. AI reads it, pulls out the fields, sets a priority, routes it to the right owner.
- Incoming requests in three languages get understood and categorised without a lookup table.
- The system drafts the reply, a human approves it in Notion, then it sends.

Does not earn it:

- AI writes a summary nobody reads.
- A chatbot that answers questions about a database you could have just looked at.
- AI generating text to fill a page so the workspace looks busy.

Rule of thumb: if an `if` statement could have done it, an `if` statement should have done it.

5. Notion's Role (read this twice, you are judged on it)

<aside>



Notion is the database, the control panel, and the audit trail. Your service can run anywhere and use any stack. But everything a human needs to see, approve, or override lives in Notion. A person who has never seen your code should be able to open the workspace and know what the system does, what it did today, and what it is waiting on.

</aside>

The test judges will apply

Turn your service off. Is the Notion workspace still a useful place to run this job?

If your workspace is a dump of JSON-looking rows nobody would read, the answer is no. If it is a clean, human-readable operations hub that your code happens to maintain, the answer is yes. Build for yes.

Common Notion mistakes (avoid these)

- Writing raw model output into pages. Format for humans: clear titles, statuses, short reasoning summaries.
- Faking the Run Log by hand. Rows written by your integration are attributed differently from rows you type. We check.
- Building the Notion layer in the last two hours. It is a judging pillar, design it on day one.

6. End Goals

By the end of the event, your project should achieve:

1. **One job, fully automated.** A trigger fires, your code does the work, a real action happens outside Notion, and a row lands in the Run Log. No human in the middle.
2. **A workspace a stranger can run the job from.** Someone who has never seen your code opens Notion and knows what happened, what is pending, and what needs them.
3. **A system that survives bad input.** Garbage in, no crash, no duplicates, nothing silently lost. Anything it cannot handle goes to a human instead of disappearing.
4. **Proof you built it across the event.** Commits and Run Log rows spread over the days, not one night.
5. **A durable design.** Ask yourselves: who notices when the system makes a bad call? If the answer is "the person reading the Notion workspace," you built the right thing.

7. Your Stack and Your Setup

- **Any stack.** Python, TypeScript, Go, or whatever you already write in. We score what your system does and how well it is built, not what it is written in.
- **Free to build.** Everything runs on Notion's free plan plus free hosting tiers.
- **Students get free Notion Education Plus** at notion.com/students, set up during the opening session.
- **Come with a boring job in mind.** The best ones come from your own week.